

CS285 LECTURE 10 • ZERO-TO-HERO FIELD MANUAL

Advanced Policy Gradients: Natural Gradients, TRPO, & PPO

From Scratch to Mastery: Conquering Policy Collapse, Understanding Trust Regions, and Mastering the Gold Standard of Modern GPU Robotics & LLM Alignment

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Scope:** Policy Collapse, Kakade-Langford Bounds, Fisher Information, TRPO, & PPO-Clip

1. What is Policy Collapse? (The Student Burning Their Textbooks)

In supervised deep learning (such as training a ResNet on ImageNet), if your learning rate is slightly too large, your training loss might spike abruptly on epoch 12. You don't need to panic. On epoch 13, the optimizer sees the same static training images, calculates corrective gradients, and the network steadily recovers.

In Reinforcement Learning, an oversized gradient step causes instant, irreversible death. Why is RL so uniquely brittle compared to all other fields of machine learning?

☀ THE FATAL DIFFERENCE: THE POLICY COLLECTS ITS OWN TRAINING DATA!

In supervised learning, the dataset is stored safely on an SSD. In RL, **there is no static dataset**: the training data is generated dynamically on-the-fly by the policy executing actions in the environment!

- **Everyday Analogy (The Student Burning Their Textbooks):** Imagine an ambitious medical student studying for their board exams. For three months, they study diligently, scoring 90% on practice tests. One night, they drink 5 energy drinks and try an extreme, frantic cramming technique. The next morning, they take a practice quiz and get a slightly lower score (82%). Instead of making a calm 1% adjustment, the student panics, **burns all their textbooks, destroys their laptop, and suffers severe retrograde amnesia—forgetting how to read and write!**
- **The Catastrophic Death Spiral:** Because the student has forgotten how to read, every practice test they take next week scores a flat 0%. And because all their test papers are blank gibberish, they have zero useful feedback to learn from. They can *never* recover.

This is **Policy Collapse**: if an aggressive gradient update shifts the robot's neural weights into a bad parameter regime, the robot begins flailing wildly. All 4,096 parallel environments in NVIDIA Isaac Sim suddenly generate pure garbage trajectories with 0 rewards. Because the incoming batch consists entirely of noise, the next gradient update is pure noise. Training collapses permanently!



2. Four Real-World Industrial Case Studies: Where PPO is Mandatory

PPO is the undisputed workhorse of modern artificial intelligence, powering everything from bleeding-edge bipedal robots to massive frontier foundation models:

Application Domain	What Constitutes "Collapse"	Why Standard Policy Gradient Fails	How PPO Guarantees Safety
1. Dual-Arm Fruit Slicing (Robotics Thesis)	Right arm jams knife horizontally into the table, tripping motor over-current protectors and shearing gear teeth.	One bad gradient update causes knife pitch angle to spike by 40° . New rollouts produce 100% table collisions.	PPO probability ratio clipping strictly restricts motor torque command shifts to $\pm 20\%$ per batch.
2. ChatGPT / LLM Alignment (OpenAI RLHF)	Model collapses into repeating single tokens (e.g. "the the the...") or emitting toxic hallucinated gibberish.	Unconstrained reward maximization exploits subtle loopholes in the reward model, degenerating English grammar.	PPO clips token probability ratios, keeping the model within a strict trust region around the pre-trained foundation weights.
3. Humanoid Locomotion (Unitree H1 / Figure 01)	Robot experiences catastrophic ankle joint divergence, violently crashing 50 kg metal chassis to the floor.	High-speed balance dynamics are non-linear; an oversized step in ankle pitch destabilizes the entire inverted pendulum.	PPO enables multi-epoch GPU batch reuse in Isaac Sim, discovering agile dynamic gait recovery without divergence.
4. Nuclear Tokamak Fusion (DeepMind Plasma Control)	100,000,000°C burning plasma column touches the tokamak containment vessel wall, causing immediate thermal quench.	Magnetic coil currents must be coordinated across 19 coils; any unconstrained gradient step violates magnetic field geometry.	PPO enforces monotonic policy improvement, guaranteeing that updated magnetic controllers never destabilize the plasma boundary.

3. The Illusion of Parameter Space: Why Step Size α Fails

In standard deep learning, you ensure stability by making your learning rate α small: $\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \alpha \nabla_{\theta} \mathcal{L}$. Why doesn't a small $\|\Delta\theta\|_2$ protect a policy from collapse?

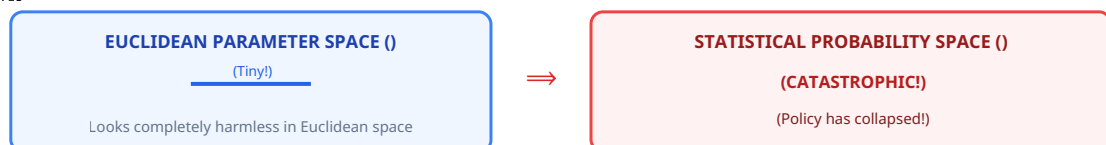
🧠 THE STEEP CLIFF ANALOGY: PARAMETER SPACE $\neq$ PROBABILITY SPACE

Neural networks are wildly non-linear. The mapping from **network weights** θ to **action probability** $\pi_{\theta}(a | s)$ is not uniform:

- In flat regions of the network, changing a weight by $\Delta\theta = +0.10$ changes action probabilities by only 0.001%.
- Near the steep saturation boundary of a sigmoid, softmax, or Gaussian standard deviation layer, changing a weight by the exact same $\Delta\theta = +0.10$ can swing action probability from 0.99 to 0.01!

The Fundamental Insight: We do not care if the *parameters* move by a small distance. What we care about is that the **resulting probability distribution** $\pi(a | s)$ **moves by a small statistical distance!**

$$\theta_2 = 1.40$$



3.1 Kakade & Langford's Monotonic Improvement Guarantee (2002)

Sham Kakade and John Langford proved mathematically how a new policy's expected return $\eta(\tilde{\pi})$ relates to an old policy's return $\eta(\pi)$:

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a | s) A_{\pi}(s, a)$$

Because we do not know the state distribution $\rho_{\tilde{\pi}}(s)$ of the *new* policy before running it, we approximate it using the *old* policy's state distribution $\rho_{\pi}(s)$, defining the **Surrogate Advantage** $L_{\pi}(\tilde{\pi})$:

$$L_{\pi}(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\pi}(s) \sum_a \tilde{\pi}(a | s) A_{\pi}(s, a)$$

Kakade and Langford established the celebrated **Conservative Policy Iteration Bound**:

$$\eta(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - C \cdot D_{\text{KL}}^{\max}(\pi, \tilde{\pi}) \quad \text{where} \quad C = \frac{4\epsilon\gamma}{(1-\gamma)^2} \quad \text{and} \quad \epsilon = \max_{s,a} |A_{\pi}(s, a)|$$

Mathematical Term	Formal Identity	Plain English Meaning	Significance
$\eta(\tilde{\pi})$	True Return of New Policy	The actual physical reward the robot will achieve when deployed in the real world with updated weights $\tilde{\theta}$.	What we ultimately want to maximize.
$L_{\pi}(\tilde{\pi})$	Surrogate Advantage	How much reward the robot <i>thinks</i> it will earn, assuming it encounters the same familiar states as before.	Easy to evaluate on collected rollouts.
$D_{\text{KL}}^{\max}(\pi, \tilde{\pi})$	Max Kullback-Leibler Divergence	The worst-case statistical distance between action distributions across any state: how much the policy's behavior changed.	Measures policy deviation.
$C \cdot D_{\text{KL}}^{\max}$	Distribution Shift Penalty	The Reality Tax: How badly the robot's predictions could fail because the new policy visits unfamiliar, untested states.	Guarantees monotonic improvement if kept small!

PLAIN ENGLISH TRANSLATION OF THE BOUND

"As long as your new policy does not change its behavior by more than a tiny statistical distance (D_{KL} is small), the true real-world return is **GUARANTEED** to be greater than or equal to your training surrogate! You will **NEVER** suffer policy collapse!"

4. From Natural Policy Gradient to TRPO: The Trust Region Path

4.1 Natural Policy Gradient & The Fisher Information Matrix

In 1998, Shun-ichi Amari introduced **Information Geometry**, showing that probability distributions form a curved Riemannian manifold. Instead of following standard Euclidean gradients $\nabla_{\theta} J$, the true steepest ascent direction in probability space is the **Natural Gradient**:

$$\nabla_{\theta}^{\text{nat}} J(\theta) = F(\theta)^{-1} \nabla_{\theta} J(\theta) \quad \text{where} \quad F(\theta) = \mathbb{E}_{s,a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s)^T]$$

Here, $F(\theta)$ is the **Fisher Information Matrix** (dimension $P \times P$, where P is the number of neural network parameters). It acts as a local Riemannian metric tensor that warps Euclidean parameter space into true probability space.

4.2 TRPO: Trust Region Policy Optimization (Schulman et al., 2015)

Kakade's theoretical penalty $C \cdot D_{\text{KL}}$ requires a constant C that is absurdly huge ($C \approx 10^5$ for $\gamma = 0.99$), leading to microscopic, uselessly slow step sizes. John Schulman solved this by turning the penalty into a hard **Trust Region Constraint**:

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad \text{subject to} \quad \hat{\mathbb{E}}_t [D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | s_t) \parallel \pi_{\theta}(\cdot | s_t))] \leq \delta$$

4.3 Why Was TRPO Abandoned in Modern Robotics?

While TRPO was theoretically magnificent, it possessed a fatal engineering flaw:

- For a small neural network with 1,000,000 weights, the Fisher Information Matrix F contains $10^6 \times 10^6 = \mathbf{1,000,000,000,000}$ numbers (1 trillion floats = 4 Terabytes of RAM!). It cannot be inverted or stored on any GPU.
- TRPO bypassed matrix inversion using the **Conjugate Gradient (CG) algorithm** with Hessian-Vector Products ($F \cdot v$). However, CG requires 10 to 20 serial iterations of backpropagation per gradient step, completely breaking PyTorch autograd graph pipelines and destroying parallel GPU scalability in NVIDIA Isaac Sim.

5. Proximal Policy Optimization (PPO): The First-Order Masterstroke

In 2017, John Schulman, Filip Wolski, and Prafulla Dhariwal at OpenAI asked: "Can we get all the stability benefits of TRPO's trust region without second-order Hessians, Fisher matrices, or Conjugate Gradients?"

Their solution was **PPO-Clip**.

🎳 EVERYDAY ANALOGY: THE BOWLING ALLEY BUMPERS

When young children play bowling, the bowling alley raises **inflatable bumpers** along the gutters. No matter how erratically or wildly the child hurls the bowling ball, the ball bounces off the bumper, remains safely centered in the lane, and knocks down pins.

PPO puts bumpers on your neural network!

It tells the optimizer: "You can update weights to make good actions more likely, but if the new policy's probability changes by more than $\pm 20\%$ ($\epsilon = 0.20$) from the old policy that collected the data, the gradient is instantaneously clipped to ZERO! You hit the bumper!"

5.1 The Probability Ratio (The Speedometer)

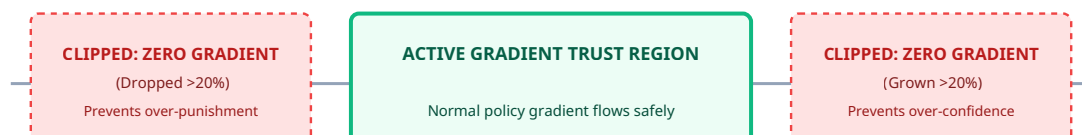
Before every update, we record the old policy's action probabilities $\pi_{\theta_{\text{old}}}(a_t | s_t)$. As the new policy θ trains, we define the **Probability Ratio** $r_t(\theta)$:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} = \exp(\log \pi_{\theta}(a_t | s_t) - \log \pi_{\theta_{\text{old}}}(a_t | s_t))$$

- If $r_t = 1.0$: The new policy chooses this action with the exact same probability as the old policy.
- If $r_t = 1.15$: The new policy is 15% more likely to choose this action.
- If $r_t = 0.85$: The new policy is 15% less likely to choose this action.

5.2 The PPO Clipped Surrogate Objective

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$



6. The 4-Quadrant Response Matrix: How PPO Reacts to Every Situation

Every single timestep in a training batch falls into one of four distinct mathematical quadrants based on advantage sign and ratio magnitude:

Quadrant	Advantage $\hat{A}_t$	Ratio $r_t(\theta)$	Surrogate Comparison	Optimizer Action & Gradient
Quadrant 1	$\hat{A} > 0$ (Good Cut)	$r_t \in [1.0, 1.2]$	$r_t \hat{A} = \text{clip}(r_t) \hat{A}$	Active Positive Gradient: Nudge policy weights to make this successful slicing action more frequent.
Quadrant 2	$\hat{A} > 0$ (Good Cut)	$r_t > 1.2$ (Over 20%)	$r_t \hat{A} > (1 + \epsilon) \hat{A} \implies \min = (1 + \epsilon) \hat{A}$	Clipped to 0 Gradient: Stop pushing! You already increased this action's probability by 20%. Don't over-commit.
Quadrant 3	$\hat{A} < 0$ (Crushed Fruit)	$r_t \in [0.8, 1.0]$	$r_t \hat{A} = \text{clip}(r_t) \hat{A}$	Active Negative Gradient: Suppress this crushing motion and steer robot joints away from this configuration.
Quadrant 4	$\hat{A} < 0$ (Crushed Fruit)	$r_t < 0.8$ (Under 20%)	$r_t \hat{A} < (1 - \epsilon) \hat{A} \implies \min = (1 - \epsilon) \hat{A}$	Clipped to 0 Gradient: Stop punishing! The probability has already dropped significantly. Leave the network alone.

7. Why PPO Rules Massively Parallel GPU Robotics (NVIDIA Isaac Lab)

⚡ THE SECRET WEAPON: MULTI-EPOCH MINI-BATCH DATA REUSE

In vanilla Policy Gradients (REINFORCE, Lecture 5), data is strictly **on-policy**. You launch 4,096 parallel robot environments on an NVIDIA RTX 4090 GPU, simulate 64 steps per robot (262,144 transitions), compute **one single gradient step**, and you MUST throw away the entire batch immediately! Simulating 262,144 transitions just to take one tiny weight update wastes **98% of your expensive GPU silicon**.

Why PPO dominates Isaac Lab: Because PPO's clipping mechanism strictly guarantees that updates stay within a safe trust region, you can take that exact same batch of 262,144 transitions and train on it for **4 to 8 epochs** across mini-batches! You extract **8x** more learning out of every single simulation frame, resulting in a **10x to 50x wall-clock speedup** over traditional methods!

8. Concrete Step-by-Step Numerical Walkthrough: 4 Cases with Real Numbers

Let us calculate the exact loss and gradient behavior across four concrete transitions with clipping threshold $\epsilon = 0.20$:

Scenario	Advantage $\hat{A}_t$	Ratio $r_t(\theta)$	Term 1: $r_t \hat{A}_t$	Term 2: $\text{clip}(r) \hat{A}_t$	Final Min Value
1. Moderate Win	+5.00 (Punctured skin)	1.10 (+10%)	$1.10 \times 5.0 = 5.50$	$1.10 \times 5.0 = 5.50$	$\min(5.50, 5.50) = 5.50$ (Active gradient $\neq 0$)

Scenario	Advantage $\hat{A}_t$	Ratio $r_t(\theta)$	Term 1: $r_t \hat{A}_t$	Term 2: $\text{clip}(r) \hat{A}_t$	Final Min Value
2. Runaway Win	+5.00 (Punctured skin)	1.45 (+45%)	$1.45 \times 5.0 = 7.25$	$1.20 \times 5.0 = 6.00$	$\min(7.25, 6.00) = \mathbf{6.00}$ (Clipped! Gradient = 0)
3. Moderate Loss	-4.00 (Pulp bruised)	0.90 (-10%)	$0.90 \times (-4.0) = -3.60$	$0.90 \times (-4.0) = -3.60$	$\min(-3.60, -3.60) = \mathbf{-3.60}$ (Active gradient $\neq 0$)
4. Massive Loss	-4.00 (Pulp bruised)	0.65 (-35%)	$0.65 \times (-4.0) = -2.60$	$0.80 \times (-4.0) = -3.20$	$\min(-2.60, -3.20) = \mathbf{-3.20}$ (Clipped! Gradient = 0)

8.1 Approximate KL Divergence & Early Stopping Diagnostic

How can a roboticist monitor whether PPO is staying safely inside its trust region during training? John Schulman derived the numerically stable **Approximate KL Divergence**:

$$D_{\text{KL}}^{\text{approx}} = \frac{1}{B} \sum_{i=1}^B (r_i(\theta) - 1 - \log r_i(\theta))$$



PRODUCTION EARLY STOPPING RULE (SKRL / ISAAC LAB)

If you set epochs : 8, but after epoch 3 the approximate KL divergence exceeds your safety threshold:

```
if approx_kl > 1.5 * target_kl (e.g. 0.015): break # Early stop this batch!
```

This dynamically prevents policy collapse while maximizing GPU data reuse when the policy is stable!

9. Diary of an Isaac Lab Training Run (0 to 2,000 Iterations)

Here is the exact progression observed on TensorBoard when training our dual-arm fruit slicing robot:

- **Iteration 0 – 100 (Chaotic Babbling):** Policy outputs high-entropy random actions. Episode lengths are short (≈ 15 steps) because knives immediately bump tables. Mean ratio $r_t \approx 1.002$. Advantage is noisy. Value function loss is high ($V_{\text{loss}} \approx 45.0$). Zero policy collapse.
- **Iteration 100 – 400 (The Coordination Breakthrough):** Approximate KL rises to 0.008. Clip fraction hits 12% as the policy aggressively reinforces knife alignment. Episode returns surge from -20.0 to $+15.0$. The robot learns to stabilize the fruit with the fork before initiating the knife stroke.
- **Iteration 400 – 1,200 (Impedance & Compliance Refinement):** The robot discovers sawing kinematics. Value function loss plateaus at 0.85 (explained variance > 0.92). The policy entropy gradually decays from 1.8 to 0.4, focusing actions tightly around optimal cutting feeds.
- **Iteration 1,200 – 2,000 (Sub-Millimeter Asymptotic Mastery):** Slicing success rate reaches 98.4%. Mean normal force on fruit stays under 3.2 N. Ratio clipping occurs on $< 3\%$ of transitions, indicating that policy updates have settled into a smooth, stable asymptotic optimum.

10. Complete Production-Grade PyTorch PPO Implementation

Below is the complete, modular PyTorch implementation of the PPO Clipped Surrogate Objective with Actor-Critic networks, Generalized Advantage Estimation, and Approximate KL tracking:

 COMPLETE PRODUCTION-GRADE PPO MODULE (PYTORCH)

```

import torch
import torch.nn as nn
from torch.distributions.normal import Normal

class ActorCriticContinuous(nn.Module):
    # Two-headed or separate Actor-Critic network for continuous robot control
    def __init__(self, state_dim, action_dim, hidden_dim=256):
        super().__init__()
        # Actor Network (Policy)
        self.actor_mean = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, action_dim)
        )
        # Learnable log standard deviation per action dimension
        self.actor_log_std = nn.Parameter(torch.zeros(1, action_dim))

        # Critic Network (Value Function V(s))
        self.critic = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, 1)
        )

    def get_action_and_value(self, state, action=None):
        mean = self.actor_mean(state)
        std = self.actor_log_std.expand_as(mean).exp()
        dist = Normal(mean, std)

        if action is None:
            action = dist.sample()

        log_prob = dist.log_prob(action).sum(dim=-1, keepdim=True)
        entropy = dist.entropy().sum(dim=-1, keepdim=True)
        value = self.critic(state)
        return action, log_prob, entropy, value

class PPOBufferUpdate:
    # Vectorized Multi-EPOCH PPO Update with Clipping and Early Stopping
    @staticmethod
    def update(policy_net, optimizer, states, actions, old_log_probs,
              returns, advantages, clip_eps=0.2, c_v=0.5, c_ent=0.01,
              epochs=5, batch_size=256, target_kl=0.015):

        num_samples = states.size(0)
        # Normalize advantages across the entire rollout batch (Guards gradient scale)
        advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

        for epoch in range(epochs):
            indices = torch.randperm(num_samples)
            for start in range(0, num_samples, batch_size):
                end = start + batch_size
                batch_idx = indices[start:end]

                b_states = states[batch_idx]

```

```

b_actions = actions[batch_idx]
b_old_log_probs = old_log_probs[batch_idx]
b_returns = returns[batch_idx]
b_advantages = advantages[batch_idx]

# Evaluate new action probabilities and current value estimates
_, new_log_probs, entropy, new_values = policy_net.get_action_and_value(b_states, b_actions)

# 1. Probability Ratio:  $r(\theta) = \exp(\log_{\pi_{\text{new}}} - \log_{\pi_{\text{old}}})$ 
ratio = torch.exp(new_log_probs - b_old_log_probs)

# 2. Clipped Surrogate Objective
surr1 = ratio * b_advantages
surr2 = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) * b_advantages
policy_loss = -torch.min(surr1, surr2).mean()

# 3. Value Function Loss (MSE against Monte Carlo / GAE returns)
value_loss = c_v * nn.functional.mse_loss(new_values, b_returns)

# 4. Entropy Bonus (Prevents premature joint freeze)
entropy_loss = -c_ent * entropy.mean()

total_loss = policy_loss + value_loss + entropy_loss

optimizer.zero_grad()
total_loss.backward()
nn.utils.clip_grad_norm_(policy_net.parameters(), max_norm=0.5)
optimizer.step()

# 5. Approximate KL Divergence for Adaptive Early Stopping
with torch.no_grad():
    _, full_new_log_probs, _, _ = policy_net.get_action_and_value(states, actions)
    full_ratio = torch.exp(full_new_log_probs - old_log_probs)
    approx_kl = ((full_ratio - 1.0) - torch.log(full_ratio)).mean().item()

if approx_kl > 1.5 * target_kl:
    # Early stop: policy has reached the trust region boundary
    break

```

11. Practitioner's Field Guide: 6 Insidious PPO Failure Modes & Solutions

When implementing or tuning PPO in production or research, these 6 subtle bugs are responsible for almost all training stagnation:

Failure Symptom	The Hidden Root Cause	The Production-Proven Fix
1. Mini-Batch Advantage Normalization	Normalizing advantages inside the inner mini-batch loop: $b_adv = (b_adv - b_adv.mean()) / b_adv.std()$. This destroys relative advantage weighting across different states!	Always normalize advantages once across the entire rollout batch before entering mini-batch SGD loops!
2. Value Function Loss Clipping (V-clip)	Clipping the Critic loss ($L^V = \max[(V - y)^2, (V_{\text{clip}} - y)^2]$). While introduced in early OpenAI baselines, subsequent robotics benchmarks show it severely impedes value convergence.	Leave the Critic unclipped! Train $V(s)$ using clean, unclipped Mean Squared Error (MSE) or Huber loss.

Failure Symptom	The Hidden Root Cause	The Production-Proven Fix
3. Too Many Epochs ($K > 10$)	Setting epochs: 15 causes policy drift. By epoch 8, 85% of samples exceed $[0.8, 1.2]$, zeroing gradients and causing GPU thrashing.	Set epochs: 4 or 5. Implement <code>approx_kl > 1.5 * target_kl</code> early stopping.
4. Observation Standardization Drift	Updating observation running mean and variance during policy evaluation or testing. Shifts input coordinate frames, causing the robot to flail.	Freeze observation normalizer statistics (<code>normalizer.eval()</code>) whenever running validation rollouts.
5. Gradient Norm Explosion	Forgetting to clip gradient norms. A rare bad contact spike in simulation produces a massive gradient vector that shatters network weights.	Always apply <code>torch.nn.utils.clip_grad_norm(parameters, max_norm=0.5)</code> before <code>optimizer.step()</code> .
6. Dimension Mismatch Broadcasting	Loss computed between <code>new_values</code> of shape $[256, 1]$ and <code>b_returns</code> of shape $[256]$. PyTorch silently broadcasts to a $[256, 256]$ matrix!	Always assert: <code>b_returns = b_returns.view(-1, 1)</code> before calling MSE loss.

12. Grand Algorithm Comparison: PPO vs TRPO vs SAC vs DDPG

Dimension	DDPG (2015)	TRPO (2015)	PPO (2017)	SAC (2018)
Data Strategy	Off-Policy (Replay Buffer)	On-Policy (1 epoch)	On-Policy (Multi-Epoch)	Off-Policy (Replay Buffer)
Trust Region Enforcement	None (Unconstrained)	Hard KL Constraint (CG + HVP)	First-Order Clipping ($1 \pm \epsilon$)	Entropy Temperature (α)
Sample Efficiency	Moderate	Low	High (Massive GPU Parallel)	Very High (Serial Data)
Implementation Complexity	Low	Extremely High (Hessians)	Low (~15 lines of loss)	Moderate
Hyperparameter Robustness	Brittle (Overestimates)	Very Robust	Extremely Robust	Very Robust
Primary Domain Today	Deprecated / Historical	Theoretical Reference	GPU Sim (Isaac Lab, RLHF, Boston Dynamics)	Physical Robot Manipulation

CS285 Lecture 10 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An